Basic (your first working tool) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

`@tool` (from `langchain_core.tools`) wraps a normal typed function into a `Tool` object that carries its name, its docstring (used as the description), and its argument shape — all derived automatically from the function's own signature and type hints. This is why type hints on tool functions aren't optional the way they might be on a throwaway script: the model literally reads them. **The docstring matters just as much** — `@tool` reads it as the tool's description shown to the model, so a missing or vague docstring is exactly the "vague description" problem the Core Concepts section warns about, even on this first, simple example.

Registering the tool on a model call — `model.bind_tools([add])` — returns a new model wrapped with that tool list attached. `bind_tools()` takes a list even for one tool, since it's the same shape you'll extend in later exercises with more tools. Calling `.invoke(...)` on it doesn't run your function; it returns an `AIMessage` whose `.tool_calls` attribute is a list of dictionaries, each with `name`, `args`, and an `id` — that's everything your code needs to actually go run the right function with the right arguments.

Actually running it: `add.invoke(response.tool_calls[0]["args"])` — note this uses `.invoke(...)` on the tool itself, not a plain Python function call, since `@tool` wraps it into a `Runnable` too.

Write out, in plain words, the two-step handoff: the model requests a call, and your code is the one that actually executes it.

Worth knowing beyond this exercise: "run the tool and print the real result" isn't actually the end of a real tool-calling round trip. The Core Concepts section's 5-step trace has 2 more steps after this — sending the result back to the model, tagged with `call["id"]`, via `ToolMessage`, and a second `.invoke(...)` call so the model produces its actual final answer. Not needed here; Doc07's agent loop is exactly that pattern, run repeatedly.

Hint 2 — The plan, and almost the whole thing

```
@tool
def add(a: int, b: int) -> int:
    """Add two integers together and return the sum."""
    return a + b

model_with_tools = ChatOpenAI(model="...", temperature=0).bind_tools([add])

response = model_with_tools.invoke("What's 5 + 7?")
print(response.tool_calls)

for call in response.tool_calls:
    if call["name"] == "add":
        result = add.invoke(call["args"])
        print(f"Real result: {result}")
```

What's missing from a real ship-it version: checking `response.tool_calls` isn't empty before looping (the model could just answer directly), and checking `call["name"]` matches what you expect instead of assuming it. Add both, then compare against the Solution.

Full solution: Show me the solution